

UNITY MINI-PROJECT BANK

UNITY MINI-PROJECT BANK

For the Unity Certified User: Programmer Exam

Each project includes a prompt, a way to infuse creativity/game concepts, and a “challenge mode”. Feel free to modify any project, adding your own creativity and twist to them (or coming up with your own project!)

Working Expectations

You may work with a partner, but you must **save your Unity project folder into a shared Google Drive every day**.

If a partner is absent and the project is not backed up, you will need to work independently on a different task for the day.

Backing up your project daily protects your work and keeps your team moving.

STARTER (20 pts) ★

Perfect for early modules (1–3) in GMetrix

0. AI-Assisted Script Writing Warm-Up

Modules: 1–2 (Unity basics, scripts, components)

Core Concept: Use an AI tool to generate a simple Unity C# script and implement it correctly.

What You Should Know Before Starting:

- How to create a new Unity scene
- How to create a GameObject (cube, sphere, capsule)
- How to create and attach a C# script
- How to save your project daily into a shared Google Drive folder

Project Overview: Practice giving clear instructions to an AI tool, copying the script into Unity, attaching it to an object, and testing it.

Creative Twist: Choose a simple behavior (rotate, move, jump, or change color) and have the AI generate the script for it.

Challenge Mode: Ask the AI to add one extra feature such as adjustable speed, a keybind toggle, or a cooldown timer.

Suggested Tools: Unity primitives, C# scripts, any AI assistant.

Support Link: See the [detailed version](#) for example prompts and troubleshooting guidance.

1. Roll-A-Ball Remix

Modules: 1–2 (Unity basics, variables, methods)

Core Concept: Move a ball with player input.

What You Should Know Before Starting:

- How to create a GameObject and add a Rigidbody
- How to write a script that reads player input
- How to use `AddForce()` to move an object

Project Overview: Build a scene where a ball moves with player input and collects items.

Creative Twist: Add collectibles with different point values.

Challenge Mode: Add a timer and a win condition.

Suggested Tools: Unity primitives, Rigidbody, UI Text.

2. Character Controller Playground

Modules: 2–3 (variables, methods, components)

Core Concept: Move a simple character around a scene.

What You Should Know Before Starting:

- How to use the CharacterController component
- How to write movement scripts using `controller.Move()`
- How to create trigger zones

Project Overview: Build a playground where a character moves and interacts with zones.

Creative Twist: Add zones that change speed, jump height, or color.

Challenge Mode: Add a stamina bar that drains when sprinting.

Suggested Tools: CharacterController, UI Slider, triggers.

3. Obstacle Course Lite

Modules: 1–3

Core Concept: Build a simple course with platforms and hazards.

What You Should Know Before Starting:

- How to build a scene with platforms and colliders
- How to detect collisions and reset position
- How to use triggers for checkpoints

Project Overview: Create a platforming course with hazards and checkpoints.

Creative Twist: Add checkpoints and a fall-reset system.

Challenge Mode: Add moving platforms using scripts.

Suggested Tools: Rigidbody, triggers, transform scripting.

4. Camera Follow Demo

Modules: 5 (Cameras, Cinemachine)

Core Concept: Make a camera follow the player smoothly.

What You Should Know Before Starting:

- How to install and use Cinemachine
- How to set a Follow target
- How to switch between virtual cameras

Project Overview: Build a camera system that follows the player and switches angles.

Creative Twist: Add multiple camera angles the player can switch between.

Challenge Mode: Add a dramatic zoom when sprinting or jumping.

Suggested Tools: Cinemachine, Input System.

POWER-UP (25 pts) ★★

Perfect for mid-course modules (4–9)

5. Endless Runner Mini

Modules: 4, 6, 7, 9

Core Concept: Build a tiny endless runner with obstacles.

What You Should Know Before Starting:

- How to move the player or environment continuously
- How to spawn obstacles
- How to update score UI

Project Overview: Create a mini endless runner with power-ups and scoring.

Creative Twist: Add power-ups (speed boost, shield, magnet).

Challenge Mode: Add procedural obstacle spawning.

Suggested Tools: Prefabs, UI Text, Random.Range.

6. NPC Dialogue Pop-Up

Modules: 2–3, 8

Core Concept: Press a key → show dialogue UI.

What You Should Know Before Starting:

- How to create and toggle UI panels
- How to trigger events with key input
- How to use TextMeshPro

Project Overview: Build a simple dialogue system triggered by player input.

Creative Twist: Add branching dialogue choices.

Challenge Mode: Add character portraits and typing animation.

Suggested Tools: Canvas, Panels, TextMeshPro.

7. Collect & Score Game

Modules: 2, 6, 8

Core Concept: Collect items → update score UI.

What You Should Know Before Starting:

- How to detect collisions
- How to update UI elements
- How to use prefabs for collectibles

Project Overview: Build a game where the player collects items and earns points.

Creative Twist: Add item rarity and sound effects.

Challenge Mode: Add a combo multiplier.

Suggested Tools: AudioSource, UI Text, prefabs.

8. Raycast Interaction Demo

Modules: 7

Core Concept: Use raycasts to interact with objects.

What You Should Know Before Starting:

- How to cast a ray from the camera
- How to detect objects with tags
- How to trigger interactions

Project Overview: Build a system where the player targets and interacts with objects.

Creative Twist: Make objects highlight when targeted.

Challenge Mode: Add a pickup and drop mechanic.

Suggested Tools: RaycastHit, materials, outline shaders.

9. Collision-Based Mini-Game

Modules: 9

Core Concept: Detect collisions to trigger events.

What You Should Know Before Starting:

- How to use colliders and Rigidbody
- How to detect `OnCollisionEnter` or `OnTriggerEnter`
- How to trigger effects or resets

Project Overview: Build a mini-game where collisions drive gameplay.

Creative Twist: Make it a tag or avoid-the-enemy game.

Challenge Mode: Add enemy AI that chases the player.

Suggested Tools: Rigidbody, colliders, NavMesh (optional).

LEGENDARY (30 pts)

Perfect for advanced modules (through #10)

10. VFX Power-Up System

Modules: 6, 10

Core Concept: Trigger particle effects + sounds when collecting items.

What You Should Know Before Starting:

- How to use Particle System and AudioSource
- How to trigger effects on collision
- How to use public variables for customization

Project Overview: Build a power-up system with unique visual and audio effects.

Creative Twist: Each power-up has a unique VFX theme.

Challenge Mode: Add cooldown timers and UI indicators.

Suggested Tools: Particle System, AudioSource, UI icons.

11. Cinematic Intro Sequence

Modules: 5, 11

Core Concept: Create a short cutscene using Timeline + Cinemachine.

What You Should Know Before Starting:

- How to use Timeline and Cinemachine
- How to animate objects and cameras
- How to sequence events

Project Overview: Build a cinematic intro that introduces a character or world.

Creative Twist: Introduce a character or world lore.

Challenge Mode: Add camera transitions synced to music.

Suggested Tools: Timeline, Cinemachine, AudioSource.

12. DOTS Swarm Simulation

Modules: 10

Core Concept: Use ECS to spawn and move many entities efficiently.

What You Should Know Before Starting:

- How to set up DOTS packages
- How to create entity prefabs
- How to write systems to move entities

Project Overview: Build a simulation of a swarm or group movement.

Creative Twist: Make it a school of fish or robot swarm.

Challenge Mode: Add simple flocking or avoidance behavior.

Suggested Tools: ECS, Jobs, Burst.

13. Mini Boss Battle

Modules: 4–12

Core Concept: Player vs. boss with health bars and attacks.

What You Should Know Before Starting:

- How to create health systems
- How to trigger attacks and damage
- How to use UI sliders for health

Project Overview: Build a boss fight with telegraphed attacks and visual effects.

Creative Twist: Add telegraphed attacks the player must dodge.

Challenge Mode: Add multiple attack phases and VFX.

Suggested Tools: Animator, UI Sliders, particle effects.

14. Micro-Open World

Modules: 4–10

Core Concept: Build a small explorable environment.

What You Should Know Before Starting:

- How to build terrain and place objects
- How to create NPCs and interactables
- How to track collectibles or quests

Project Overview: Build a small open world with secrets and exploration.

Creative Twist: Add NPCs, secrets, and world-building details.

Challenge Mode: Add a minimap and quest markers.

Suggested Tools: Terrain tools, UI, NavMesh.

15. Full Endless Runner (Advanced)

Modules: 4–10

Core Concept: Build a polished endless runner with UI, VFX, and audio.

What You Should Know Before Starting:

- How to build a runner loop
- How to spawn obstacles and power-ups
- How to polish with UI and effects

Project Overview: Build a complete endless runner with unlockables and terrain generation.

Creative Twist: Add unlockable characters or skins.

Challenge Mode: Add procedural terrain generation.

Suggested Tools: Prefabs, Randomization, ScriptableObjects (optional).

16. Build-Your-Own Game Project

Modules: Any (you choose based on your idea)

Core Concept: Design and build a custom Unity project that fits your interests and aligns with certification skills.

What You Should Know Before Starting:

- How to scope a project so it's achievable.
 - Think about:
 - What are the most essential things that **MUST** be included? (Starter level)
 - What are the things that would be nice to have but are not essential? (Power-Up level)
 - What would be really cool to include but might be the most complicated or difficult to figure out? (Legendary level)
- How to identify the core mechanic your game depends on
- How to save your project daily into a shared Google Drive folder

Project Overview: Pitch and build your own Unity mini-game or interactive experience. You will propose a concept, outline the core mechanic, list the features you want to include, and map out how the project will grow across three skill levels. Your teacher will help you refine the scope and align it with certification-relevant skills.

Creative Twist: Draw inspiration from elements you've seen in other projects, such as:

- Movement systems (platformer, top-down, physics-based)
- Collecting, scoring, or inventory mechanics
- Dialogue, interaction, or raycast-based systems
- Camera systems, VFX, or UI-driven gameplay
- Enemy behavior, hazards, or simple AI

Choose any combination that excites you and fits your idea.

Challenge Mode: Add a mastery-level feature that pushes your skills, such as procedural generation, advanced UI, multi-phase enemy behavior, or a polished cinematic sequence.

Suggested Tools: Any Unity systems or modules you've learned so far.

Support Link: See the [detailed version](#) for pitch templates, scoping tips, and example roadmaps.

Help: #0 AI Script Writing

0. AI-Assisted Script Writing Warm-Up

(Detailed Version with Helper Prompts)

A guided practice project to help you learn how to use an AI tool to generate a simple Unity C# script.

Project Overview:

You will use an AI tool of your choice (Copilot, ChatGPT, Gemini, etc.) to generate a very small Unity script. The goal is to practice giving clear instructions, copying the script into Unity, attaching it to a GameObject, and testing it. This project ensures you can follow instructions, troubleshoot, and save your work correctly before moving on to larger projects.

What You Should Already Know:

- How to create a new Unity scene
- How to create a GameObject (cube, sphere, capsule)
- How to create a new C# script and attach it to an object
- How to save your project into a shared Google Drive folder daily

Mini-Lesson / Concept Reminder:

AI tools can help you write scripts, but **you must give clear instructions**.

A good AI prompt includes:

- What the script should do
- What object it will be attached to
- How the player interacts with it
- Any variables you want to customize

You are still responsible for reading the script, understanding it, and testing it.

Starter Tasks (First 10 Minutes):

- Open a new Unity scene
- Create a simple object (cube or sphere)
- Open your AI tool
- Ask it to generate a script that makes the object move, rotate, or change color
- Copy the script into Unity
- Attach it to your object
- Press Play and test it

Creative Twist:

Choose one of these simple goals for your AI-generated script:

- Make the object rotate at a speed you can adjust
- Make the object move back and forth
- Make the object change color when you press a key
- Make the object jump when you press space

You choose the behavior — the AI writes the script.

Challenge Mode:

Ask the AI to add one extra feature, such as:

- A public speed variable
- A keybind to toggle the effect on/off
- A cooldown timer
- A color cycle instead of a single color change

Suggested Assets / Tools:

Unity primitives, C# script editor, AI tool of your choice (Copilot, Gemini, Claude, etc.).

Example AI Prompts (If You Get Stuck)

Here are some prompts you can copy/paste into the AI tool if you are stuck:

Basic Prompt:

"Write a simple Unity C# script that I can attach to a cube. The script should make the cube rotate at a customizable speed. Include comments so I can understand what each part does."

Movement Prompt:

"Write a Unity C# script that moves a sphere back and forth between two points. Use public variables so I can adjust the speed and distance in the Inspector."

Color Change Prompt:

"Write a Unity C# script that changes the object's color when I press the spacebar. Use `GetComponent<Renderer>()` and `Input.GetKeyDown`."

Troubleshooting Prompt:

"I'm getting an error in Unity. Here is the script and the error message. Explain what the error means and how to fix it."

Refinement Prompt:

"Modify this script so that the speed is a public variable and I can change it in the Inspector."

Help: #16 Build Your Own Game

16. Build Your Own Project

Modules:

Any (you choose based on your idea)

Core Concept:

Design and build a custom Unity project that fits your interests and aligns with certification skills.

What You Should Know Before Starting

How to scope a project so it's achievable.

Think about:

- **Starter level:** What are the essential features your game *must* have to function?
- **Power-Up level:** What features would be nice to have but are not required for the game to work?
- **Legendary level:** What features would be really cool but might be the most complex or time-consuming?

How to identify the core mechanic your game depends on.

Your core mechanic is the single action or system the entire game revolves around.

Examples:

- Moving and jumping
- Collecting items
- Avoiding hazards
- Talking to NPCs
- Interacting with objects
- Solving puzzles
- Switching cameras
- Fighting enemies

How to save your project daily into a shared Google Drive folder.

If you are working with a partner, this is required so you can continue progress even if someone is absent.

Project Overview

Pitch and build your own Unity mini-game or interactive experience. You will propose a concept, outline the core mechanic, list the features you want to include, and map out how the project will grow across three skill levels. Your teacher will help you refine the scope and align it with certification-relevant skills. This project is meant to be creative, flexible, and personalized. You are encouraged to combine ideas from previous projects or invent something entirely new.

Creative Twist: Inspiration for Your Idea

You may draw inspiration from elements you've seen in other projects, such as:

Movement Systems

- Platformer movement
- Top-down movement
- Physics-based rolling or bouncing
- Dash, sprint, or jump mechanics

Collecting, Scoring, or Inventory

- Collect items to earn points
- Gather resources to unlock areas
- Inventory with item rarity
- Timed collection challenges

Dialogue, Interaction, or Raycasts

- NPC conversations
- Inspecting objects
- Opening doors or activating switches
- Picking up and dropping items

Camera, VFX, or UI-Driven Gameplay

- Cinemachine camera switching
- Dramatic zooms or camera shake
- Health bars, timers, or score UI
- Particle effects for power-ups or hits

Enemy Behavior, Hazards, or Simple AI

- Enemies that chase the player
- Patrol paths
- Avoid-the-enemy gameplay
- Boss attacks or telegraphed moves

Choose any combination that excites you and fits your idea.

Challenge Mode

Add a mastery-level feature that pushes your skills, such as:

- Procedural generation
- Advanced UI (inventory, minimap, quest log)
- Multi-phase enemy behavior
- A polished cinematic sequence
- Environmental effects or dynamic lighting
- ScriptableObjects for data management

Suggested Tools

Any Unity systems or modules you've learned so far, including:

- Rigidbody or CharacterController
- UI Toolkit or Canvas UI
- Cinemachine
- Timeline
- Particle System
- Raycasting
- NavMesh
- ScriptableObjects
- Prefabs and object pooling

Planning Your Project: Step-By-Step

1. Choose Your Concept

Write a one-sentence description of your game.

Examples:

- "A top-down game where you collect ingredients before time runs out."
- "A platformer where you avoid hazards and reach the goal."
- "A short cinematic story introducing a character."
- "A mini boss battle with telegraphed attacks."

2. Identify Your Core Mechanic

What does the player do most often?

This should be the center of your design.

3. List Your Features

Sort them into three categories:

Starter (must-have):

The smallest version of your game that still works.

Power-Up (nice-to-have):

Adds depth or polish but isn't required.

Legendary (stretch goals):

Ambitious features that require advanced skills.

4. Map Out Your Roadmap

Write a simple plan for how your project will grow:

Starter:

- Core mechanic working
- Basic environment

- One or two essential features

Power-Up:

- Additional systems
- UI improvements
- More interactions or enemies

Legendary:

- Advanced mechanics
- VFX, audio, polish
- Cinematics or procedural elements

5. Identify the Modules Your Project Uses

Examples:

- Movement → Modules 2-3
- UI → Module 8
- Cameras → Module 5
- VFX → Module 10
- AI → Module 9
- DOTS → Module 12

6. Pitch Your Idea

Your pitch should include:

- Title
- Concept
- Core mechanic
- Feature list
- Starter → Power-Up → Legendary roadmap
- Module alignment
- Why you want to make this project

Your teacher will help you refine the scope.

Examples of Possible Projects

Example A: “Crystal Runner” (Collect & Avoid Game)

Core Mechanic: Move and collect crystals while avoiding hazards.

Starter: Player movement, 5 crystals, simple hazards.

Power-Up: Item rarity, score multiplier, sound effects.

Legendary: Enemy AI that chases the player, VFX for rare items.

Example B: “Echo Forest” (Exploration + Dialogue)

Core Mechanic: Explore and talk to NPCs.

Starter: Player movement, one NPC, simple dialogue.

Power-Up: Multiple NPCs, branching dialogue, small quests.

Legendary: Cinematic intro, environmental storytelling, minimap.

Example C: “Sky Arena” (Mini Boss Battle)

Core Mechanic: Dodge boss attacks and deal damage.

Starter: Player movement, boss with one attack.

Power-Up: Health bars, telegraphed attacks, sound cues.

Legendary: Multi-phase boss, VFX, dramatic camera transitions.

Troubleshooting Tips

- If your idea feels too big, reduce the number of features.
- Build the Starter version completely before adding anything else.
- Test every new feature before moving on.
- Ask your AI tool for help writing or fixing scripts.
- If you get stuck, focus on your core mechanic first.